چیت‌شیت نوشتن کد ماشین — همهٔ نکته‌ها از ساده تا پیشرفته

ساختار و زبان کامپیوتر — دکتر سربازی‌آزاد — برای سؤال «ترجمه به کد ماشین» و «این برنامه چه می‌کند؟». همهٔ جدول‌های مثال را همان اسمبلر آزموده‌شدهٔ کاتالوگ برنامه‌ها ساخته است.

سطح ۱ پایه | سطح ۲ آدرس و داده | سطح ۳ پرش و بیت‌های حالت | سطح ۴ پیشرفته | نمونهٔ کامل و وارسی

۱. الگوریتم کلی: ترجمه در ۱۰ گام ۱
۲. سطح ۱: خواندن مشخصات ماشین ۱
۳. طول قالب‌ها بر حسب بیت و واحد ۱
۴. سطح ۱: انتخاب کد عمل و قالب از روی شکل عملوندها ۲
۵. سطح ۱: ترتیب فیلدها با ترتیب عملوندها فرق دارد ۳
۶. سطح ۲: عددها داخل فیلدها ۴
۷. سطح ۲: شمارندهٔ مکان (LC) و جدول نمادها ۴
۸. سطح ۲: عبارت‌های آدرس، # و پرانتز ۵
۹. سطح ۲: داده‌ها (dw) ۵
۱۰. سطح ۳: پرش‌ها (مطلق، نسبی، غیرمستقیم) ۶
۱۱. سطح ۳: بیت‌های حالت، بیت‌های ثابت و فیلدهای بلااستفاده ۶
۱۲. سطح ۳: نوشتن جواب (هگز، اکتال یا دودویی؟) ۶
۱۳. سطح ۴: آدرس یک فیلد داخل دستور (کد خودتغییرده) ۷
۱۴. سطح ۴: رمزگشایی معکوس (کد ماشین ← اسمبلی) ... ۷
۱۵. نمونهٔ کامل ۱: ماشین ۵ (فیلدهای ۴ و ۶ بیتی، واحد بایتی) ۸
۱۶. نمونهٔ کامل ۲: میان‌ترم ۱۴۰۲ سؤال ۴ (واحد ۶ بیتی) ۸
۱۷. وارسی نهایی و خطاهای رایج ۸

۱ الگوریتم کلی: ترجمه در ۱۰ گام

1. **مشخصات ماشین** را بیرون بکش: طول واحد آدرس‌پذیر u، طول کلمه w و k = w/u، بیت‌های آدرس، تعداد ثبات‌ها، Big/Little Endian، نمایش اعداد (بخش ۲).
2. **طول هر قالب** را هم بر حسب بیت و هم بر حسب **واحد** کنار جدول بنویس (بخش ۳).
3. **گذر ۱:** با شمارندهٔ مکان (LC) آدرس هر خط را حساب کن و جدول نمادها را بساز (بخش ۷).
4. برای هر دستور از روی نیمونیک **و شکل عملوندها** (`#`، `( )`، `Rn`، `A(Rn)`) کد عمل و قالب را انتخاب کن (بخش ۴).
5. مقدار هر فیلد را پیدا کن: شمارهٔ ثبات، آدرس از جدول نمادها، داده، بیت‌های حالت، جابه‌جایی (بخش‌های ۸ تا ۱۱).
6. هر مقدار را به دودویی **با عرض دقیق همان فیلد** تبدیل کن: صفر از چپ، و برای منفی مکمل دو در همان عرض (بخش ۶).
7. فیلدها را **به ترتیب قالب** (نه ترتیب عملوندها در اسمبلی!) از چپ به راست کنار هم بگذار (بخش ۵).
8. رشتهٔ بیتی را **واحدبه‌واحد** بنویس: واحد ۴ بیتی یک رقم هگز، ۸ بیتی یک بایت، ۶ بیتی دو رقم اکتال (بخش ۱۲).
9. `dw` ها را هم کلمه‌به‌کلمه بنویس (Big Endian، منفی با مکمل دو، `?` نامعلوم) (بخش ۹).
10. **وارسی:** تعداد بیت‌ها، آدرس آخر، و یک بار رمزگشایی معکوس هر دستور (بخش ۱۷).

۲ سطح ۱: خواندن مشخصات ماشین

۱ چه چیزهایی را از صورت سؤال برداریم

مورد	فرمول / نکته	مثال
واحد آدرس‌پذیر u	هر آدرس به یک واحد اشاره می‌کند	۴، ۶، ۷، ۸ یا ۱۶ بیت
طول کلمه w	k = w / u واحد در هر کلمه	۱۴۰۲: ۲۴/۶ = ۴
بیت‌های آدرس	$\log_2$(واحدها#)	2^{21} واحد ⇐ ۲۱ بیت
حافظه بر حسب کلمه	k × واحدها = #کلمه‌ها	۱۳۹۸: 2^{29}×8 = 2^{32}
فیلد ثبات	$\log_2$(ثبات‌ها#)	۶ ⇐ R0..R63 بیت
ترتیب بایت	همهٔ ماشین‌های درس Big Endian	بیت/واحد پرارزش اول
هم‌ترازی	ناهم‌تراز ⇐ کلمه از هر آدرسی شروع می‌شود	داده بلافاصله بعد از کد
نمایش اعداد	مکمل دو؛ ممیز شناور IEEE 754	-1 = همه یک
ثبات‌های خاص	R0 = 0، آدرس بازگشت در R31 یا خانهٔ ثابت	۱۳۹۸، ۱۳۹۹، ۱۴۰۰، ۱۴۰۱

۱ توان‌های ۲ پرکاربرد

2^n	dec	hex	used in
2^{7}	128	80h	`M1`
2^{12}	4,096	1000h	`M4`
2^{14}	16,384	4000h	`1400 words`
2^{16}	65,536	10000h	`M2,M3,M5,1A,1400`
2^{21}	2,097,152	200000h	`1401`
2^{24}	16,777,216	1000000h	`1399,1402`
2^{29}	536,870,912	20000000h	`1398 words`
2^{32}	4,294,967,296	100000000h	`M6,M7,1398`

۱ طول فیلد آدرس در قالب معمولاً با تعداد بیت آدرس برابر است؛ اگر کوتاه‌تر بود (مثل addr12 در ماشین ۴ با 2^{12} واحد) همان کافی است. اگر آدرس یک برچسب در فیلد جا نشد، برنامه با آن قالب قابل ترجمه نیست.

۲ اگر ماشین چند قالب دارد، **کد عمل معمولاً طول دستور را هم تعیین می‌کند**؛ پس قبل از هر کاری جدول «کد عمل ⇐ قالب ⇐ طول» را بساز.

۳ طول قالب‌ها بر حسب بیت و واحد

Machine	unit	word	formats (bits = units)
M1	16	16 = 1u	—: 16b = 1u
M2	8	16 = 2u	—: 40b = 5u
M3	8	16 = 2u	—: 40b = 5u
M4	4	16 = 4u	I: 12b = 3u II: 20b = 5u III: 24b = 6u
M5	8	16 = 2u	I: 16b = 2u II: 32b = 4u III: 32b = 4u
1A	4	16 = 4u	—: 20b = 5u
M6	8	32 = 4u	I: 40b = 5u II: 8b = 1u
M7	8	32 = 4u	I: 40b = 5u II: 16b = 2u
1398	4	32 = 8u	I: 40b = 10u II: 12b = 3u III: 48b = 12u
1399	6	24 = 4u	I: 18b = 3u II: 12b = 2u III: 36b = 6u V: 30b = 5u
1400	4	16 = 4u	I: 36b = 9u II: 20b = 5u III: 52b = 13u
1401	7	21 = 3u	I: 49b = 7u II: 28b = 4u
1402	6	24 = 4u	I: 54b = 9u II: 30b = 5u

قاعده‌ها

۳ u ÷ طول دستور (واحد) = بیت‌های قالب. در طراحی درست همیشه بخش‌پذیر است؛ اگر نبود، دوباره جمع بیت‌ها را حساب کن.

۴ LC بعد از هر دستور به اندازهٔ **طول همان قالب** جلو می‌رود؛ دستورهای یک برنامه می‌توانند طول‌های متفاوت داشته باشند (مثلاً ۱۴۰۰: ۵، ۹ یا ۱۳ واحد).

۵ بلندترین قالب = IR، MBR = w، بیت‌های آدرس = MAR = PC؛ برای سؤال ۱ هم همین جدول کافی است.

۶ `dw` همیشه k واحد برای هر مقدار می‌گیرد (۱۴۰۲: ۴، ۱۴۰۱: ۳، ۱۳۹۸: ۸، ماشین ۷: ۴).

۴ سطح ۱: انتخاب کد عمل و قالب از روی شکل عملوندها

شکل عملوند ⇐ حالت آدرس‌دهی

#x	بی‌واسطه: خودِ مقدار x (اگر x برچسب است، **آدرس** آن)
x	مستقیم: محتوای خانهٔ x
(x)	غیرمستقیم حافظه‌ای: خانه‌ای که آدرسش در x است
Rn	ثباتی مستقیم
(Rn)	ثباتی غیرمستقیم: $M_{(Rn)}$
A(Rn)	شاخص‌دار: $M_{A+(Rn)}$؛ آدرس A در فیلد addr و n در فیلد ثبات

۷ یک نیمونیک، چند کد عمل: در ماشین ۳ `mov` سه کد دارد (00، 01، 02) و `add a,(b)` کد 05 است؛ در ماشین ۴ `mov` هشت کد دارد؛ در ماشین تک‌آدرسه `load #` = 0، `load` = 2، `load ( )` = 9.

۸ بیت حالت به‌جای کد جدا: در ۱۴۰۲/۱۴۰۱/۱۴۰۰ کد عمل ثابت است و بیت‌های `d` ، `i` ، `x y z` حالت را می‌گویند.

۹ کد عمل گسترش‌پذیر: ۶ + M5: 1111 بیت؛ ۱۳۹۸: 1111 + ۴ بیت؛ ۱۳۹۹: پیشوند 1 / 011 / 010 / 00 قالب را تعیین می‌کند.

۱۰ دو دستور با یک کد: با یک بیت جدا می‌شوند: ۱۴۰۰ `inc/dec` و `jmp/call` با x؛ ۱۴۰۲ `jz/jnz` و `jneg/jmp` با x؛ ۱۴۰۱ قالب I و II با xy = 11.

ماشین ۵: سه قالب در یک برنامه

Addr	Assembly	Fields	Code (bytes)	نکته
0000	`xor R0,R0`	0101 000000 000000	50 00	I
0002	`add R1,#2`	1111000001 000001 0002h	F0 41 00 02	III
0006	`jne R1,R2,L`	1011 000001 000010 000Ah	B0 42 00 0A	II
000A	`L: mov R3,A(R1)`	0110 000011 000001 000Eh	60 C1 00 0E	II
000E	`A: dw 0`	0000h	00 00	

ماشین ۱۳۹۸: پیشوند 0000 / 1111 / op (هر رقم هگز یک واحد)

Addr	Assembly	Fields	Code (hex digit = 4b)	نکته
00000000	`mov R2,#3F800000h`	0000 0010 3F800000h	023F800000	I
0000000A	`fadd R3,R2`	1010 0011 0010	A32	II
0000000D	`mov R1,x(R0)`	1111 0000 0001 0000 0000003Dh	F0100000003D	III 0000
00000019	`mov x(R0),R3`	1111 0001 0011 0000 0000003Dh	F1300000003D	III 0001
00000025	`L: loop- R1,R4,L`	1111 0010 0001 0100 00000025h	F21400000025	III 0010
00000031	`fjh R2,R6,L`	1111 1001 0010 0110 00000025h	F92600000025	III 1001
0000003D	`x: dw 0`	00000000h	00000000	

ماشین ۱۴۰۲: همهٔ حالت‌ها و هر چهار پرش (واحد ۶ بیتی ⇐ کد به اکتال)

Addr	Assembly	Fields	Code (octal pair = 6b)	نکته
000000	`mov x,#5`	000 000041h 010 000005h	00 00 00 10 12 00 00 00 05	yz=10 data
000009	`mov x,y`	000 000041h 000 000045h	00 00 00 10 10 00 00 01 05	yz=00
000012	`mov x,(p)`	000 000041h 001 000049h	00 00 00 10 11 00 00 01 11	yz=01
00001B	`mov (p),y`	000 000049h 100 000045h	00 00 00 11 14 00 00 01 05	x=1
000024	`add x,#-1`	101 000041h 010 FFFFFFh	50 00 00 10 12 77 77 77 77	data = FFFFFF
00002D	`jz L`	110 00003Ch 011	60 00 00 07 43	110 x=0
000032	`jnz L`	110 00003Ch 111	60 00 00 07 47	110 x=1
000037	`jneg L`	111 00003Ch 011	70 00 00 07 43	111 x=0
00003C	`L: jmp L`	111 00003Ch 111	70 00 00 07 47	111 x=1
000041	`x: dw 0`	000000h	00 00 00 00	
000045	`y: dw 0`	000000h	00 00 00 00	
000049	`p: dw x`	000041h	00 00 01 01	

ماشین ۱۴۰۱: بیت i، بیت‌های xy و قالب II با xy = 11 (واحد ۷ بیتی)

Addr	Assembly	Fields	Code (hex per 7b unit)	نکته
000000	`mov a,#5`	0000 0 000025h 10 000005h	00 00 01 16 00 00 05	xy=10
000007	`cmp (p),a`	0100 1 000028h 00 000025h	24 00 01 20 00 00 25	i=1 xy=00
00000E	`add a,#-2`	0110 0 000025h 10 1FFFFEh	30 00 01 16 7F 7F 7E	21-bit: 1FFFFE
000015	`inc a`	0001 0 000025h 11	08 00 01 17	F II: xy=11
000019	`dec (p)`	0010 1 000028h 11	14 00 01 23	i=1
00001D	`L: jmp (1FFFFDh)`	0110 1 1FFFFDh 11	37 7F 7F 77	i=1 indirect
000021	`call L`	0101 0 00001Dh 11	28 00 00 77	
000025	`a: dw 0`	000000h	00 00 00	
000028	`p: dw a`	000025h	00 00 25	

ماشین ۱۴۰۰: بیت d، بیت x و قالب سه‌آدرسه (هر رقم هگز یک واحد)

Addr	Assembly	Fields	Code (hex digit = 4b)	نکته
0000	mov a,#5	000 0040h 1 0005h	000810005	d=1
0009	add a,b	001 0040h 0 0044h	200800044	d=0
0012	inc a	011 0040h 0	60080	011 x=0
0017	dec a	011 0040h 1	60081	011 x=1
001C	L: jmp (FFFEh)	100 FFFEh 0	9FFFC	100 x=0
0021	call L	100 001Ch 1	80039	100 x=1
0026	je a,#0,L	101 0040h 1 0000h 001Ch	A00810000001C	III d=1
0033	jl a,b,L	111 0040h 0 0044h 001Ch	E00800044001C	III d=0
0040	a: dw 0	0000h	0000	
0044	b: dw 0	0000h	0000	

ماشین ۱۳۹۹: چهار قالب با پیشوند (واحد ۶ بیتی)

Addr	Assembly	Fields	Code (octal pair = 6b)	نکته
000000	mov R1,R2	00 0000 0 00001 0 00010	00 01 02	I
000003	add (R1),R2	00 0001 1 00001 0 00010	01 41 02	I i1=1
000006	swap (R1),(R2)	00 0110 1 00001 1 00010	06 41 42	I i1=i2=1
000009	addq R1,4	00 0111 0 00001 000100	07 01 04	I qdata
00000C	inc4 R1	010 0110 00001	23 01	II
00000E	jmp R31	010 0000 11111	20 37	II
000010	L: loop R3,L	011 1000 00011 000010h	34 03 00 00 00 20	III
000016	mov R5,#100	1 00101 000064h	45 00 00 01 44	V

۵ سطح ۱: ترتیب فیلدها با ترتیب عملوندها فرق دارد

۱۱ **مهم‌ترین تلهٔ ترجمه:** فیلدها را همیشه به ترتیب **شکل قالب** پر کن، نه به ترتیب نوشته‌شدن عملوندها. در `mov A,R3` ماشین ۴، ثبات R3 دوم نوشته شده ولی فیلد r قبل از فیلد addr است: `6 3 01D` و نه `6 01D 3`. هر جا قالب «op | r | addr» است، ثبات همیشه قبل از آدرس می‌آید.

ماشین ۴: هشت شکل mov

Addr	Assembly	Fields	Code (hex digit = 4b)	نکته
000	mov R3,A	0101 0011 01Dh	5301D	op 5 : r ← (M)
005	mov A,R3	0110 0011 01Dh	6301D	op 6 : M ← r
00A	mov R3,(A)	0111 0011 01Dh	7301D	op 7
00F	mov (A),R3	1000 0011 01Dh	8301D	op 8
014	mov R3,(R2)	0001 0011 0010	132	op 1 (قالب I)
017	mov R3,#A	1101 0011 001Dh	D3001D	op D : خودِ آدرس A
01D	A: dw 7	0007h	0007	

ماشین ۵: r1 همیشه ثبات داده است

Addr	Assembly	Fields	Code (bytes)	نکته
0000	mov R3,A(R1)	0110 000011 000001 0014h	60 C1 00 14	op 6 : r1=R3 r2=R1
0004	mov A(R1),R3	0111 000011 000001 0014h	70 C1 00 14	op 7 : r1=R3 r2=R1
0008	mov R2,R5,A	1000 000010 000101 0014h	80 85 00 14	op 8 : r1=R2 r2=R5
000C	mov A,R2,R5	1001 000010 000101 0014h	90 85 00 14	op 9 : r1=R2 r2=R5
0010	call R0,0(R63)	1110 000000 111111 0000h	E0 3F 00 00	op E
0014	A: dw 0	0000h	00 00	

`mov A(R1),R3` و `mov R3,A(R1)` فقط در کد عمل (6 و 7) فرق دارند؛ r1 = R3 و r2 = R1 (شاخص) در هر دو.

ماشین ۱۳۹۹: جهت انتقال در کد عمل است، نه در ترتیب فیلدها

Addr	Assembly	Fields	Code (octal pair = 6b)	نکته
000000	mov R2,x	011 0000 00010 00001Dh	30 02 00 00 00 35	III 0000
000006	mov x,R2	011 0001 00010 00001Dh	30 42 00 00 00 35	III 0001
00000C	mov R2,(x)	011 0010 00010 00001Dh	31 02 00 00 00 35	III 0010
000012	mov (x),R2	011 0011 00010 00001Dh	31 42 00 00 00 35	III 0011
000018	mov R2,#x	1 00010 00001Dh	42 00 00 00 35	V
00001D	x: dw 0	000000h	00 00 00 00	

۶ سطح ۲: عددها داخل فیلدها

۱۲ **عرض دقیق:** هر مقدار را با تعداد بیت همان فیلد و صفرهای پیشرو بنویس: آدرس 1Dh در فیلد ۲۴ بیتی = 00000000000000000011101 = 00001Dh.

۱۳ **شمارهٔ ثبات به دودویی:** R10 در فیلد ۴ بیتی = 1010 (هگز A)، در فیلد ۶ بیتی = 001010. R63 = 111111.

۱۴ **منفی‌ها:** مکمل دو **در عرض همان فیلد**: `#-1` در ۱۶ بیت FFFFh، در ۲۱ بیت 1FFFFFh، در ۲۴ بیت FFFFFFh. `#-2` در ۲۱ بیت = 1FFFFEh (نه FEh).

۱۵ **علامت‌گذاری عددها:** `#100h` یا `#0100` = 256. عدد هگزی که با حرف شروع می‌شود معمولاً با 0 نوشته می‌شود (`0ABCDh`).

۱۶ **بزرگ‌تر از فیلد:** اگر مقدار در عرض فیلد جا نشود (مثلاً `addq` با داده بیش از ۶ بیت در ۱۳۹۹) آن قالب قابل استفاده نیست؛ از قالب دیگر (مثلاً `mov r,#data` ۲۴ بیتی) استفاده کن.

۱۷ **تفریق با منفی:** `sub x,#FFFFFFh` یعنی x + 1 = x - (-1)؛ (ترفند ۱۴۰۲) `add R5,#-1` همان `sub R5,#1` است ولی کد متفاوت دارد:

Addr	Assembly	Fields	Code (hex digit = 4b)	نکته
000	`add R5,#-1`	1110 0101 FFFFh	E5FFFF	-1 در ۱۶ بیت
006	`sub R5,#1`	1111 0101 0001h	F50001	همان نتیجه
00C	`mov R6,#-200`	1101 0110 FF38h	D6FF38	FF38h

Addr	Assembly	Fields	Code (hex digit = 4b)	نکته
0000	`add #-1`	0001 FFFFh	1FFFF	1 FFFF
0005	`load #-4`	0000 FFFCh	0FFFC	0 FFFC

هگز ↔ دودویی و اکتال

hex	bin	hex	bin
0	0000	8	1000
1	0001	9	1001
2	0010	A	1010
3	0011	B	1011
4	0100	C	1100
5	0101	D	1101
6	0110	E	1110
7	0111	F	1111

oct	bin
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

مکمل دو در عرض‌های مختلف (هگز)

value	8 bit	12 bit	16 bit	21 bit	24 bit	32 bit
-1	FF	FFF	FFFF	1FFFFF	FFFFFF	FFFFFFFF
-2	FE	FFE	FFFE	1FFFFE	FFFFFE	FFFFFFFE
-3	FD	FFD	FFFD	1FFFFD	FFFFFD	FFFFFFFD
-4	FC	FFC	FFFC	1FFFFC	FFFFFC	FFFFFFFC
-8	F8	FF8	FFF8	1FFFF8	FFFFF8	FFFFFFF8
-10	F6	FF6	FFF6	1FFFF6	FFFFF6	FFFFFFF6
-16	F0	FF0	FFF0	1FFFF0	FFFFF0	FFFFFFF0
-100	9C	F9C	FF9C	1FF9C	FFF9C	FFFFF9C
-128	80	F80	FF80	1FF80	FFF80	FFFFF80

روش سریع: $2^n - |x|$؛ یا همهٔ بیت‌ها را معکوس کن و ۱ اضافه کن؛ یا از راست تا اولین ۱ را نگه دار و بقیه را معکوس کن.

۷ سطح ۲: شمارندهٔ مکان (LC) و جدول نمادها

۱۸ `org X` ⇐ LC = X (کد ماشین ندارد). اگر `org` نیامده باشد LC = 0.

۱۹ برچسب هر خط = LC **قبل** از آن خط. برچسب روی `dw` = آدرس اولین کلمه.

۲۰ بعد از هر خط: طول قالب LC += (دستور) یا LC += n × k (dw با n مقدار؛ `N dup(v)` یعنی N مقدار).

۲۱ **دو گذر:** چون برچسب‌ها ممکن است بعداً تعریف شوند (ارجاع رو به جلو)، اول همهٔ آدرس‌ها را حساب کن و بعد کد بساز.

۲۲ `end` و توضیح‌ها (بعد از `;`) کد ندارند. خط فقط‌برچسب (بدون دستور) هم طول صفر دارد.

۲۳ نمادی که در برنامه تعریف نشده (مثل `OS` یا `OS_Control_Point`) را با `????` بنویس و بگو «آدرس آن را سیستم‌عامل/پیونددهنده تعیین می‌کند».

نمونه: جمع آرایه در ماشین ۳ (هر دستور ۵ بایت، هر کلمه ۲ بایت)

LC	line	size	symbol
100	`loop: add sum,(arrptr)`	5	`loop = 100h`
105	`add arrptr,two`	5	
10A	`sub count,two`	5	
10F	`jnz count,loop`	5	
114	`arrptr: dw array`	2	`arrptr = 114h`
116	`sum: dw 0`	2	`sum = 116h`
118	`count: dw 200`	2	`count = 118h`
11A	`two: dw 2`	2	`two = 11Ah`
11C	`array: dw 100 dup(?)`	200	`array = 11Ch`

Addr	Assembly	Fields	Code (bytes)	نکته
0100	`loop: add sum,(arrptr)`	00000101 0116h 0114h	05 01 16 01 14	غیرمستقیم
0105	`add arrptr,two`	00000011 0114h 011Ah	03 01 14 01 1A	عنصر بعد
010A	`sub count,two`	00000100 0118h 011Ah	04 01 18 01 1A	
010F	`jnz count,loop`	10000000 0118h 0100h	80 01 18 01 00	۱۰۰ دور
0114	`arrptr: dw array`	011Ch	01 1C	
0116	`sum: dw 0`	0000h	00 00	
0118	`count: dw 200`	00C8h	00 C8	
011A	`two: dw 2`	0002h	00 02	
011C	`array: dw 100 dup(?)`	????h ????h ????h … ×100	?? ?? ?? ?? ?? ?? ?? ?? …	

۸ سطح ۲: عبارت‌های آدرس، # و پرانتز

۲۴ همهٔ عبارت‌ها بر حسب واحد: `label+k` یعنی k واحد بعد از برچسب، نه k کلمه. `A+400` = ۱۰۰ کلمه × ۴ واحد؛ ماشین ۷: `array+396` = ۹۹ کلمه × ۴؛ ۱۴۰۱: `c-3` = ۱۴۰۰ یک کلمه قبل؛ ۱۳۹۹: `loop-6` = ۶ واحد قبل (ممکن است وسط یک دستور باشد!).

۲۵ `#A` با `A` فرق دارد: `mov R0,#A` آدرس A را در فیلد داده می‌گذارد (اشاره‌گر می‌سازد)؛ `mov R0,A` آدرس A را در فیلد آدرس می‌گذارد و در اجرا محتوای A خوانده می‌شود. کد عمل یا بیت حالت هم فرق می‌کند (ماشین ۴: D در برابر 5).

۲۶ `عدد#` در دستورهای آرایه: گام اشاره‌گر = k (نه ۱): `add p,#4` در ۱۴۰۲، `add ptr,#3` در ۱۴۰۱، `inc4` در ۱۳۹۹.

۲۷ شاخص‌دار با ثبات صفر = مستقیم: `x(R0)` در ۱۳۹۸ (R0 همیشه صفر) و `0(R63)` در M5 = «آدرس داخل R63».

۲۸ مقدار ثابت **بزرگ‌تر از یک کلمه** یا منفی در عبارت، باید در عرض فیلد مقصد خلاصه شود (پیمانهٔ 2^n).

نتیجهٔ چند عبارت (واقعی از امتحان‌ها)

expression	machine	value
B-396 (B = 2E1h)	1400	2E1h - 18Ch = 155h
A+400 (A = 151h)	1400	151h + 190h = 2E1h = B
#B-8	1400	data = 2D9h (A[98])
here+14 (here = 12h)	1402	20h
s-4 (s = 108h)	1402	104h = c+4
c-3 (c = 103h)	1401	100h = a
loop-6 (loop = 13h)	1399	0Dh (mid-instruction)
array-8(R1)	1398	addr = 10h - 8 = 8
#array+396	1399	data = array + 18Ch

۹ سطح ۲: داده‌ها (dw)

۲۹ هر مقدار = یک کلمه = k واحد؛ **Big Endian**: واحد پرارزش در آدرس کوچک‌تر.

۳۰ منفی با مکمل دو در عرض کلمه: `dw -1` = FFFFFFFFh (۳۲ بیت)، FFFFh (۱۶ بیت).

۳۱ `dw label` و `dw label+8` = **آدرس** (اشاره‌گر)؛ در جابه‌جایی برنامه باید اصلاح شود.

۳۲ `dw 3,4,5` سه کلمهٔ پشت‌سرهم؛ `dw 2 dup(?)` دو کلمهٔ نامعلوم (????)؛ `dw 100 dup(0)` صد کلمهٔ صفر.

۳۳ ثابت ممیز شناور: بیت‌های IEEE را بنویس (1.0 = = 2^{-10}، 3F800000h). در امتحان ۹۸ `-1.0e+109` یعنی -1.0×2^{109} ⇐ E = 109+127 = 236 ⇐ F6000000h.

۳۴ واحدهای کوچک‌تر از کلمه: در ماشین‌های ۴ بیتی هر کلمهٔ ۱۶ بیتی ۴ رقم هگز است (`dw 7` ⇐ 0007)؛ در ۶ بیتی هر کلمه ۴ جفت اکتال (`dw 77` ⇐ 01 00 00 15).

ماشین ۷ (کلمهٔ ۳۲ بیتی، بایتی)

Addr	Assembly		Fields	Code (bytes)	نکته
00000100	a: dw	-1	FFFFFFFFh	FF FF FF FF	FFFFFFFF
00000104	b: dw	80000000h	80000000h	80 00 00 00	کوچک‌ترین عدد
00000108	p: dw	arr	00000110h	00 00 01 10	آدرس arr
0000010C	q: dw	arr+8	00000118h	00 00 01 18	آدرس arr[2]
00000110	arr: dw	3,4,5	00000003h 00000004h 00000005h	00 00 00 03 00 00 00 04 …	سه کلمه پشت‌سرهم
0000011C	t: dw	2 dup(?)	????????h ????????h	?? ?? ?? ?? ?? ?? ?? ??	نامعلوم

ماشین ۱۳۹۸ (ثابت‌های ممیز شناور)

Addr	Assembly		Fields	Code (hex digit = 4b)	نکته
00000000	one: dw	3F800000h	3F800000h	3F800000	1.0
00000008	eps: dw	3A800000h	3A800000h	3A800000	2^-10
00000010	f: dw	-1.0e+109	F6000000h	F6000000	-1.0 × 2^109
00000018	g: dw	-40.5e+112	FA220000h	FA220000	-40.5 × 2^112
00000020	i: dw	-1	FFFFFFFFh	FFFFFFFF	صحیح

۱۰ سطح ۳: پرش‌ها (مطلق، نسبی، غیرمستقیم)

۳۵ **پرش مطلق:** فیلد آدرس = آدرس برچسب مقصد (بر حسب واحد)، نه شمارهٔ دستور.

۳۶ **پرش نسبی:** مقصد - (آدرس دستور پرش + طول آن) = d؛ چون هنگام اجرا PC به دستور بعد اشاره می‌کند. منفی ⇐ مکمل دو در عرض فیلد (۸ بیت: 100h - |d|).

۳۷ **بازه:** با ۸ بیت $-128 \le d \le 127$. اگر بیرون بود از پرش مطلق (`jmpl`) در ماشین ۷ استفاده کن.

۳۸ **پرش غیرمستقیم / ثباتی:** `jmp (FFFEh)` (۱۴۰۰) و `jmp (1FFFFDh)` (۱۴۰۱) آدرس خانه‌ای را می‌گیرند که آدرس مقصد در آن است (بیت x یا 1 = i)؛ `jmp R31` (۱۳۹۹) شمارهٔ ثبات را.

۳۹ **call** آدرس دستور **بعد** (PC افزایش‌یافته) را ذخیره می‌کند: در ثبات (`call r,addr` ،۱۳۹۹ `call r1,addr(r2)` M5) یا خانهٔ ثابت (FFFEh در ۱۴۰۰، 1FFFFDh در ۱۴۰۱).

۴۰ **پرش‌های شرطی دوعملوندی** (ماشین ۳: `jnz count,loop`) فیلد اول = متغیر شرط و فیلد دوم = مقصد؛ سه‌عملوندی (۱۴۰۰: `jne a,#d,L` ،M5: `jne R1,R2,L`) آدرس مقصد در آخرین فیلد است.

۴۱ در ماشین پشته‌ای (۶) پرش فیلد ندارد: مقصد قبلاً با `push #label` روی پشته گذاشته می‌شود (`push #loop` ⇐ 00 00000000).

ماشین ۷: نسبی به جلو، نسبی به عقب و مطلق

Addr	Assembly	Fields	Code (bytes)	نکته
00000100	`L: load x`	00000000 00000113h	00 00 00 01 13	
00000105	`jpos F`	10000011 00000111	83 07	d = F - 107h = 7
00000107	`subi 1`	00000111 00000001h	07 00 00 00 01	
0000010C	`jmp L`	10000100 11110010	84 F2	d = 100h - 10Eh = -14
0000010E	`F: jmpl far`	00001000 00000300h	08 00 00 03 00	مطلق: دور است
00000113	`x: dw -3`	FFFFFFFDh	FF FF FF FD	
00000300	`far: load x`	00000000 00000113h	00 00 00 01 13	

jump	at	next PC	target	d	code
jpos F	105	107	10E	+7	83 07
jmp L	10C	10E	100	-14 = F2	84 F2
jpos continue (sort)	128	12A	15E	+34h	83 34
jmp loop (sort)	15C	15E	100	-5Eh = A2	84 A2
jnz loop (sort)	168	16A	100	-6Ah = 96	80 96
loop → next (primes)	18A	18C	100	-140 ✗	jmpl

۱۱ سطح ۳: بیت‌های حالت، بیت‌های ثابت و فیلدهای بلااستفاده

۴۲ **بیت‌های حالت را از تعریف جدول بخوان** و همان ترتیب را رعایت کن: ۱۴۰۲ «x y z d»، ۱۴۰۰ «i … xy»، ۱۴۰۱ «i1 r1 i2 r2 (بعد از addr1)»، ۱۳۹۹ «i1 r1 i2 r2».

۴۳ **بیت‌های ثابت قالب** را دقیقاً بنویس: ۱۴۰۲ قالب II ،x11 ۱۴۰۱ قالب II ،11، ماشین ۵ قالب III ،1111 ۱۳۹۸ قالب III ،1111 ۱۳۹۹ پیشوندها.

۴۴ **فیلد بلااستفاده = صفر** (مثلاً عملوندهای `nop` در ماشین ۱) مگر صورت سؤال چیز دیگری بگوید.

۴۵ **عملوند تک‌آدرسه‌ها:** در ماشین تک‌آدرسه و ۷ عملوند دوم همیشه ACC است و در کد نمی‌آید؛ در ماشین ۶ جز push/pop هیچ فیلدی نیست (یک بایت).

۴۶ **دستورهای یک‌عملوندی در قالب دوعملوندی** (`inc/dec/clr/not/neg` ۱۴۰۱): فقط i و addr1 پر می‌شوند و xy = 11.

جای بیت‌های حالت در هر ماشین امتحانی

machine	layout	meaning
1402 I	op3 a1 | x y z | a2	x: opr1 ind; yz 00 dir 01 ind 10 imm
1402 II	op3 addr | x 1 1	x picks jz/jnz, jneg/jmp
1401 I	op4 i a1 | xy | a2	i: opr1 ind; xy as 1402, 11 → II
1401 II	op4 i a1 | 1 1	i: target / opr1 indirect
1400 I/III	op3 a1 d a2 (a3)	d=1: a2 field is data
1400 II	op3 a1 x	inc/dec, jmp(ind)/call
1399 I	00 op4 i1 r1 i2 r2	i=1: M[(r)]; q-forms: 6-bit data
1398	— (no mode bits)	mode = format / R0 trick

۱۲ سطح ۳: نوشتن جواب (هگز، اکتال یا دودویی؟)

۴۷ **همیشه از دودویی فیلدبه‌فیلد شروع کن** و بعد آن را بر حسب واحد گروه کن. این هم قالب را نشان می‌دهد و هم اشتباه ترتیب را می‌گیرد.

۴۸ **نمایش بر حسب واحد:** واحد ۴ بیتی ⇐ یک رقم هگز برای هر واحد؛ ۸ بیتی ⇐ یک بایت هگز؛ ۶ بیتی ⇐ دو رقم **اکتال**؛ ۷ بیتی ⇐ دودویی (یا دو رقم هگز برای هر واحد)؛ ۱۶ بیتی ⇐ چهار رقم هگز.

۴۹ **هگز کردن کل دستور وقتی بیت‌ها مضرب ۴ نیست اشتباه است:** دستور ۱۸ بیتی ۱۳۹۹ را نمی‌شود چهار رقم هگز نوشت؛ سه واحد اکتال بنویس (`mov R1,R2` ⇐ 01 00 02).

۵۰ **گروه‌بندی از چپ (MSB)** چون Big Endian است؛ اولین واحد در آدرس شروع دستور قرار می‌گیرد.

۵۱ **شکل جدول جواب:** آدرس | کد ماشین (فیلدها) | کد بر حسب واحد | برچسب و دستور اسمبلی. جدول نمادها و فرض‌ها (آدرس OS، مقدار `?`) را زیر جدول بنویس.

یک دستور، سه نمایش (۱۴۰۲ `add c+4, s-4`)

fields	101 | 00000000000000000100000100 | 000 | 00000000000000000100000100
6-bit units	101000 000000 000000 100000 100000 000000 000000 000100 000100
octal	50 00 00 40 40 00 00 04 04

واحد ۴ بیتی (۱۴۰۰ `jmp (FFFEh)`)

fields	100 | 1111111111111110 | 0
4-bit units	1001 1111 1111 1111 1100
hex	9FFFC

در ۱۴۰۰ مرز فیلدها با مرز واحدها یکی نیست (op سه بیتی)، پس رقم‌های هگز با مقدار فیلدها فرق دارند: آدرس FFFE به‌صورت «FFFC» دیده می‌شود. برای همین اول فیلدها را دودویی بنویس.

۱۳ سطح ۴: آدرس یک فیلد داخل دستور (کد خودتغییرده)

۵۲ کلمه‌ای که از واحد k دستور X شروع می‌شود (`X+k`) بیت‌های [k·u, k·u + w) دستور را می‌پوشاند.

۵۳ اگر فیلد کامل داخل این کلمه باشد و e بیت پایین‌تر از فیلد در کلمه باقی بماند، برای افزودن d به فیلد باید $d \cdot 2^e$ به کلمه اضافه کنی (اگر e = 0 همان d).

۵۴ اگر فیلد از اول کلمه بیرون بزند (پوشش جزئی)، فقط بیت‌های پایین فیلد عوض می‌شوند؛ تا وقتی نقلی به بیت‌های بیرون نرسد درست است (۱۴۰۰: `add Arr+10,#8` ⇐ addr1 + 4).

۵۵ XOR با یک ثابت روی کلمهٔ دستور، بیت‌های همان محدوده را برمی‌گرداند (۱۴۰۱: 402h ⊕ 43Dh = 03Fh ⇐ `add a,#-2` به `jmp next` تبدیل می‌شود).

۵۶ در ترجمه، `loop+3` فقط یک عبارت آدرس است (مقدار loop + 3) و مثل هر آدرس دیگری در فیلد قرار می‌گیرد.

Addr	Assembly	Fields	Code (bytes)	نکته
0100	`loop: add sum,array`	00000011 010Ah 010Eh	03 01 0A 01 0E	addr2 = loop+3
0105	`add loop+3,two`	00000011 0103h 010Ch	03 01 03 01 0C	addr2 += 2
010A	`sum: dw 0`	0000h	00 00	
010C	`two: dw 2`	0002h	00 02	
010E	`array: dw 0`	0000h	00 00	

Addr	Assembly	Fields	Code (octal pair = 6b)	نکته
000000	`loop: add sum,A`	101 000012h 000 000016h	50 00 00 02 20 00 00 00 26	addr2 = units 5..8
000009	`add loop+5,#4`	101 000005h 010 000004h	50 00 00 00 52 00 00 00 04	addr2 += 4
000012	`sum: dw 0`	000000h	00 00 00 00	
000016	`A: dw 0`	000000h	00 00 00 00	

جدول آماده: هر فیلد آدرس/داده در کدام کلمه است (X = آدرس شروع دستور)

پوشش	×	word at	bits	field	F	M
کامل	1	**X+1**	8..23	addr1	—	M2
کامل	1	**X+3**	24..39	addr2	—	M2
کامل	1	**X+1**	8..23	addr1	—	M3
کامل	1	**X+3**	24..39	addr2	—	M3
کامل	1	**X+1**	8..19	addr	II	M4
کامل	1	**X+2**	8..23	data	III	M4
کامل	1	**X+2**	16..31	addr	II	M5
کامل	1	**X+2**	16..31	data	III	M5
کامل	1	**X+1**	4..19	addr/data	—	1A
کامل	1	**X+1**	8..39	addr/data	I	M6
کامل	1	**X+1**	8..39	addr/data	I	M7
کامل	1	**X+2**	8..39	data	I	1398
کامل	1	**X+4**	16..47	addr	III	1398
کامل	1	**X+2**	12..35	addr	III	1399
کامل	1	**X+1**	6..29	data	V	1399
فقط ۱۵ بیت پایین	2^1 = 2	**X+1**	3..18	addr1	I	1400
کامل	1	**X+5**	20..35	addr2/data	I	1400
فقط ۱۵ بیت پایین	2^1 = 2	**X+1**	3..18	addr1	II	1400
فقط ۱۵ بیت پایین	2^1 = 2	**X+1**	3..18	addr1	III	1400
کامل	1	**X+5**	20..35	addr2/data	III	1400
کامل	1	**X+9**	36..51	addr3	III	1400
فقط ۱۹ بیت پایین	2^2 = 4	**X+1**	5..25	addr1	I	1401
کامل	1	**X+4**	28..48	addr2/data	I	1401
فقط ۱۹ بیت پایین	2^2 = 4	**X+1**	5..25	addr1	II	1401
فقط ۲۱ بیت پایین	2^3 = 8	**X+1**	3..26	addr1	I	1402
کامل	1	**X+5**	30..53	addr2/data	I	1402
فقط ۲۱ بیت پایین	2^3 = 8	**X+1**	3..26	address	II	1402

همهٔ سطرهای این جدول با محاسبهٔ بیتی بررسی شده‌اند (افزودن 2^e به کلمه دقیقاً یک واحد به فیلد اضافه می‌کند).

۱۴ سطح ۴: رمزگشایی معکوس (کد ماشین ← اسمبلی)

گام‌ها (برای سؤال ۳ و برای وارسی سؤال ۴)

1. از آدرس PC، بیت‌ها را واحدبه‌واحد به دودویی بنویس.
2. از روی **بیت‌های اول** قالب را تشخیص بده (پیشوند، کد عمل یا بیت‌های ثابت مثل xy = 11).
3. به اندازهٔ طول همان قالب بیت بردار و فیلدها را جدا کن.
4. کد عمل + بیت‌های حالت ⇐ نیمونیک و شکل عملوندها؛ آدرس‌ها را با جدول نمادها به برچسب برگردان.
5. PC را به اندازهٔ طول قالب جلو ببر و تکرار کن. **مرز dwها مهم نیست:** دستور ممکن است از وسط یک کلمه شروع شود یا دو کلمه را بپوشاند.

تشخیص قالب: ۱۳۹۹: …1 ۱۳۹۸ III، 011 II، 010 I، 00 V: نیبل اول 0 ⇐ F، I، بقیه II. ۱۴۰۱/۱۴۰۲: III ⇐ 111–101، II ⇐ 100–011، I ⇐ 010–000: بیت‌های حالت عملوند دوم 11 ⇐ III ⇐ 1111، I ⇐ 0101–0000 :M5. بقیه II.

نمونه: `dw 41006h` در ۱۳۹۹

Addr	Assembly	Fields	Code (octal pair = 6b)	نکته
000000	dw 41006h	041006h	01 01 00 06	این «داده» اجرا می‌شود

bits	decode
000001 000001 000000	00 | 0001 | 0 00001 | 0 00000 ⇒ add R1,R0
000110 + next 2 units	00 | 0110 | … ⇒ swap (instruction continues into next word)

به همین روش در ۱۳۹۸ دو کلمهٔ اول برنامه (F6000000 و 03300000) با هم یک دستور ۱۲ واحدی `je R0,R0,330h` می‌سازند.

۱۵ نمونهٔ کامل ۱: ماشین ۵ (فیلدهای ۴ و ۶ بیتی، واحد بایتی)

LC: 100 xor (I، ۲) ⇐ 102 mov (I، ۲) ⇐ 104 mov #200 (III، ۴) ⇐ 108 loop (II، ۴) ⇐ 10C add (I) ⇐ 10E add #2 (III) ⇐ 112 jne (II) ⇐ 116 mov sum,R0,R0 (II) ⇐ 11A call (II) ⇐ 11E sum ⇐ 120 A. **نمونهٔ کدگذاری:** `mov R3,A(R1)` ⇐ `000001 | 000011 | 0110 0000 1100 0001` = 60C1h و بعد آدرس A = 0120h.

Addr	Machine code	Fields (instruction format)	Label	Assembly		توضیح
				`org`	`100h`	
0100	50 00	0101 000000 000000		`xor`	`R0,R0`	انباره = 0
0102	00 40	0000 000001 000000		`mov`	`R1,R0`	شاخص = 0
0104	F0 02 00 C8	1111000000 000010 00C8h		`mov`	`R2,#200`	شاخص پایان
0108	60 C1 01 20	0110 000011 000001 0120h	`loop:`	`mov`	`R3,A(R1)`	شاخص‌دار
010C	10 03	0001 000000 000011		`add`	`R0,R3`	
010E	F0 41 00 02	1111000001 000001 0002h		`add`	`R1,#2`	
0112	B0 42 01 08	1011 000001 000010 0108h		`jne`	`R1,R2,loop`	
0116	90 00 01 1E	1001 000000 000000 011Eh		`mov`	`sum,R0,R0`	ذخیره
011A	E0 3F 00 00	1110 000000 111111 0000h		`call`	`R0,0(R63)`	بازگشت به OS
011E	00 00	0000h	`sum:`	dw	0	
0120	?? ?? ?? ?? ?? ?? ?? ?? …	????h ????h ????h … ×100	`A:`	dw	100 dup(?)	
				`end`		

نمادها: loop = 108h, sum = 11Eh, A = 120h

۱۶ نمونهٔ کامل ۲: میان‌ترم ۱۴۰۲ سؤال ۴ (واحد ۶ بیتی)

LC: mov@0 ⇐ and@9 ⇐ here@18 (12h) ⇐ sub@23 ⇐ add@32 (20h) ⇐ jnz@41 ⇐ jz@46 ⇐ org 100h: c = 100h (دو کلمه)، s = 108h. **عبارت‌ها:** here+14 = 20h، s-4 = 104h، c+4 = 104h، s+4 = 10Ch. **نمونه:** `sub s,#FFFFFFh` ⇐ `111111111111111111111111 | 010 | 00000000000000100001000` = 100.

Addr	Machine code	Fields (instruction format)	Label	Assembly		توضیح
				`org`	`0`	
000000	00 00 00 41 40 00 00 04 04	000 00010Ch 000 000104h		`mov`	`s+4, c+4`	
000009	10 00 00 41 40 00 00 04 00	001 00010Ch 000 000100h		`and`	`s+4, c`	
000012	60 00 00 04 03	110 000020h 011	`here:`	`jz`	`here+14`	
000017	40 00 00 41 02 77 77 77 77	100 000108h 010 FFFFFFh		`sub`	`s, #FFFFFFh`	
000020	50 00 00 40 40 00 00 04 04	101 000104h 000 000104h		`add`	`c+4, s-4`	
000029	60 00 00 00 07	110 000000h 111		`jnz`	`here-18`	
00002E	?? ?? ?? ?? ??	110 ??????h 011		`jz`	`OS_Control_Point`	
				`org`	`100h`	
000100	?? ?? ?? ?? 00 00 00 01	??????h 000001h	`c:`	dw	?, 1	
000108	00 00 00 00 ?? ?? ?? ??	000000h ??????h	`s:`	dw	0, ?	

نمادها: here = 12h, c = 100h, s = 108h

۱۷ وارسی نهایی و خطاهای رایج

چک‌لیست قبل از تحویل

- ☐ تعداد بیت هر دستور = طول قالب (و مضرب u).
- ☐ آدرس هر خط = آدرس قبلی + طول خط قبلی؛ آدرس برچسب‌ها با جدول نمادها یکی است.
- ☐ کد عمل با **شکل عملوند** (#، ( )، ثابت) می‌خواند.
- ☐ ترتیب فیلدها مثل شکل قالب است.
- ☐ عددهای منفی در عرض فیلد/کلمه مکمل دو شده‌اند.
- ☐ پرش‌های نسبی از دستور بعد حساب شده‌اند و در بازه‌اند.
- ☐ بیت‌های ثابت (1111، 11، x11)، پیشوندها) نوشته شده‌اند.
- ☐ dwها: هر مقدار k واحد، Big Endian، ? و نمادهای بیرونی مشخص.
- ☐ یک دستور تصادفی را معکوس رمزگشایی کن؛ باید همان سطر اسمبلی برگردد.

خطاهای رایج

خطا	غلط	درست
شمردن کلمه به‌جای واحد	`A+100`	`A+400 (k = 4)`
ترتیب فیلد = ترتیب عملوند	`mov A,R3 ⇒ 6 01D 3`	`6 3 01D`
منفی بدون گسترش علامت	`#-2 ⇒ 0000FE`	`1FFFFE (21b)`
نسبی از خود پرش	`d = 10E - 105`	`d = 10E - 107`
فراموش کردن #	`mov R0,A ⇒ 5…`	`mov R0,#A ⇒ D…`
هگز کل دستور غیرمضرب ۴	`18b ⇒ 0102h`	`00 01 02 (oct)`
صفرهای پیشرو	`addr = 1Dh (5b)`	`00001Dh (24b)`
شمارهٔ ثبات دهدهی	`R10 ⇒ 0001 0000`	`1010`
طول ثابت برای همهٔ دستورها	`LC += 9`	`I 9 / II 5 / III 13`
dw چندمقداری یک کلمه	`dw 3,4,5 ⇒ 4u`	`12u (3 × 4)`
جا انداختن بیت ثابت	`inc a ⇒ op i a1`	`op i a1 11`
شمارهٔ دستور به‌جای آدرس	`jnz loop ⇒ 4`	`آدرس loop`
نوشتن Little Endian	`0102h ⇒ 02 01`	`01 02`
آدرس OS حدسی	`0000`	`???? + توضیح`